

SecureSign: Plataforma Web de Firma Digital y Validación Criptográfica Integrada con DevSecOps

Chavez Kleber

Carrera de Ingeniería de Software
Universidad de las Fuerzas
Armadas ESPE
Sangolquí, Ecuador
kechavez2@espe.edu.ec

Orrico Camilo

Carrera de Ingeniería de Software
Universidad de las Fuerzas
Armadas ESPE
Sangolquí, Ecuador
caorrico@espe.edu.ec

Toscano Jossue

Carrera de Ingeniería de Software
Universidad de las Fuerzas
Armadas ESPE
Sangolquí, Ecuador
jatoscano6@espe.edu.ec

Resumen—En este trabajo se presenta SecureSign, una plataforma web cliente-servidor orientada a la protección de documentos digitales, cuya propuesta combina SHA-256, cifrado AES-256-GCM, firmas RSA-PSS y certificados X.509 generados por una autoridad certificadora simulada. Asimismo, incorpora autenticación con tokens de duración limitada, contraseñas derivadas con PBKDF2, permisos según el rol y el propietario, persistencia relacional y registro de auditoría. Para sustentar las decisiones del proyecto se realizó una revisión rápida de literatura basada en las fases de planificación, conducción y reporte de Kitchenham, mientras que la evaluación incluyó 140 pruebas automatizadas, análisis estático y 30 repeticiones con un archivo de 1 081 344 bytes. Los tiempos medios fueron 3.074 ms para el cifrado, 3.008 ms para el descifrado y 1.516 ms para la verificación de firmas; además, todas las alteraciones provocadas durante las pruebas fueron detectadas y tanto Bandit como la auditoría de dependencias del frontend finalizaron sin hallazgos. En conjunto, estos resultados indican que es posible reunir controles criptográficos y prácticas DevSecOps en una aplicación académica de tamaño reducido; sin embargo, para utilizarla en un entorno real todavía serían necesarios TLS, gestión externa de secretos y mecanismos más seguros para almacenar las claves.

Index Terms—DevSecOps, firma digital, criptografía aplicada, AES-GCM, RSA-PSS, certificados X.509, revisión rápida.

I. INTRODUCCIÓN

Cuando se intercambian documentos mediante una aplicación web no basta con almacenarlos y permitir su descarga, pues también se debe evitar que personas no autorizadas los lean, comprobar que no hayan sido modificados y relacionar cada operación con un usuario. Estas necesidades corresponden a la confidencialidad, la integridad y la autenticidad; no obstante, si la seguridad se revisa únicamente al final del desarrollo, varios problemas pueden descubrirse demasiado tarde. Por esta razón, DevSecOps propone incorporarla durante todo el proceso y repartir esa responsabilidad entre quienes participan en el proyecto, mediante prácticas como la automatización, el enfoque *shift-left* y la evaluación continua [1]–[3].

El objetivo del trabajo fue diseñar, implementar y evaluar una plataforma reproducible que reúna la gestión de usuarios, documentos y certificados con varias operaciones criptográficas. El proyecto aporta tres elementos principales: una arquitectura web funcional con autorización y auditoría, una síntesis de literatura guiada por Kitchenham y un conjunto de resultados

experimentales y de seguridad que puede volver a ejecutarse desde el repositorio.

II. MARCO TEÓRICO

II-A. Controles criptográficos

SHA-256 genera resúmenes de 256 bits y forma parte del estándar Secure Hash Standard [4]; en SecureSign se utiliza para identificar el contenido de un archivo y comprobar si este cambió. Sin embargo, no se usa directamente para guardar contraseñas, ya que una función rápida no ofrece suficiente protección frente a intentos masivos fuera de línea. Para ese caso se eligió PBKDF2 con sal aleatoria y 600 000 iteraciones, debido a que esta función deriva una clave a partir de la contraseña, una sal y un número de iteraciones, lo que incrementa el costo de cada intento de adivinación [5], [6].

AES es el algoritmo de cifrado simétrico normalizado en FIPS 197 [7] y, para este proyecto, se trabajó con el modo GCM porque combina el cifrado con la autenticación de los datos, de acuerdo con la recomendación SP 800-38D de NIST [8]. En cuanto a las firmas, se empleó RSASSA-PSS con SHA-256, un esquema probabilístico descrito en PKCS #1 [9]. Por otra parte, los certificados X.509 permiten asociar una clave pública con un sujeto mediante la firma de una autoridad certificadora; además, el perfil definido en RFC 5280 contempla la vigencia, la construcción de la ruta de confianza y la información de revocación [10]. Para mantener el alcance académico del prototipo, estos elementos se implementaron de forma simplificada.

II-B. DevSecOps

Myrbakken y Colomo-Palacios observaron que varios controles tradicionales de seguridad no seguían el mismo ritmo de trabajo que DevOps [1]; más adelante, Rajapakse *et al.* revisaron 54 estudios y organizaron los principales desafíos en personas, prácticas, herramientas e infraestructura [2]. En este sentido, la adopción no depende solamente de instalar herramientas, sino que también requiere conocimientos compartidos y una cultura de seguridad dentro del equipo [11], [12]. Otros trabajos relacionan estos desafíos con prácticas y métricas concretas [13], mientras que la evidencia industrial resalta el análisis estático, las pruebas automatizadas y la responsabilidad compartida [3]. A partir de estas ideas se

definió una canalización que ejecuta pruebas, compila el sistema y revisa tanto el código como las dependencias.

III. METODOLOGÍA

III-A. Revisión rápida según Kitchenham

La revisión se organizó con las fases de planificación, conducción y reporte propuestas por Kitchenham y Charters [14]. Antes de iniciar la búsqueda se plantearon dos preguntas:

- **RQ1:** ¿Qué prácticas y herramientas facilitan incorporar seguridad continua en una aplicación web criptográfica?
- **RQ2:** ¿Qué controles permiten verificar integridad, autenticidad y confidencialidad de documentos?

La búsqueda se realizó el 26 de julio de 2026 en los índices web de IEEE Xplore, ACM DL, ScienceDirect y SpringerLink, además de algunos repositorios académicos. Se utilizó la siguiente cadena:

```
(DevSecOps OR "secure software development")
AND (practice OR tool OR challenge)
AND (cryptography OR "digital signature" OR
certificate)
```

Se consideraron trabajos en inglés o español publicados entre 2016 y 2026. Para ingresar a la revisión debían contar con revisión por pares, texto o metadatos verificables y una relación directa con las preguntas planteadas. Se descartaron duplicados, material publicitario, documentos sin un método identificable y estudios alejados de la seguridad durante el ciclo de desarrollo.

La búsqueda inicial produjo 14 candidatos y, después de retirar dos duplicados, se revisaron 12 títulos y resúmenes. De estos documentos, tres tenían poca relación con el tema y uno no mostraba evidencia suficiente sobre su método, por lo que la selección final quedó en ocho estudios. Para valorar su calidad se asignó un punto por cada uno de estos criterios: pregunta de investigación explícita, método descrito, evidencia rastreable y discusión de limitaciones; de esta manera, se aceptaron los trabajos que alcanzaron al menos tres de los cuatro puntos. Como los métodos y las métricas no eran comparables entre todos los estudios, se realizó una síntesis narrativa y se conservaron la cadena, los criterios y las decisiones para que el proceso pudiera revisarse posteriormente, tal como sugieren las guías de Kitchenham [14], [15].

La Tabla I resume los temas que aparecieron con mayor frecuencia en los estudios seleccionados: automatización, responsabilidad compartida, integración de herramientas, dificultades culturales y uso de métricas [1]–[3], [11], [13].

III-B. Desarrollo y evaluación

El desarrollo se organizó mediante un *backlog* incremental, mientras que la verificación se dividió en tres niveles: pruebas unitarias de las primitivas criptográficas, una prueba integral de la API y la compilación del cliente. En el experimento se cifró, descifró, firmó y verificó un archivo fijo de 1 081 344 bytes durante 30 repeticiones, cuyos tiempos se midieron con un reloj monotónico de alta resolución. La media se obtuvo mediante

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i, \quad (1)$$

Cuadro I
SÍNTESIS DE LA REVISIÓN DE LITERATURA

Tema	Evidencia sintetizada
Prácticas	<i>Shift-left</i> , automatización, seguridad compartida, evaluación continua y trazabilidad.
Herramientas	SAST, análisis de dependencias, pruebas automáticas, escaneo de contenedores e infraestructura como código.
Desafíos	Complejidad de integración, falsos positivos, cultura, competencias y balance entre velocidad y seguridad.
Métricas	Vulnerabilidades, cobertura, tiempo de operación, tasa de detección y mitigación.

mientras que la capacidad de detección se calculó como la proporción de firmas rechazadas después de modificar un byte del archivo. Finalmente, el análisis de seguridad incluyó Bandit para el backend y npm audit para las dependencias de producción del frontend.

IV. ARQUITECTURA E IMPLEMENTACIÓN

La aplicación se divide en una interfaz desarrollada con React y una API REST construida con FastAPI, mientras que para el acceso a datos se emplea SQLAlchemy: PostgreSQL se utiliza en los contenedores y SQLite en las pruebas locales. A su vez, la autenticación usa JWT e incluye una fecha de expiración, campo previsto por la especificación para impedir que un token siga siendo aceptado después del tiempo establecido [16]. De forma general, el sistema contiene los siguientes componentes:

- **Identidad:** registro, inicio de sesión, JWT HS256 con expiración, PBKDF2 y roles administrador/usuario.
- **Documentos:** carga restringida a 5 MB, consulta, descarga, borrado, SHA-256, firma y verificación.
- **Criptografía:** AES-256-GCM con nonce aleatorio, RSA-OAEP para cifrado de claves y RSA-PSS para firmas.
- **PKI simulada:** emisión X.509, consulta, verificación de vigencia y confianza, y revocación persistente.
- **Auditoría:** eventos de acceso, operaciones, fallos y validaciones sin almacenar contraseñas, claves o texto plano.

La autorización se comprueba en el servidor, de manera que un usuario común solo puede operar sobre sus propios registros, mientras que el administrador tiene acceso a la consulta de usuarios y eventos de auditoría. Por otra parte, las entradas estructuradas se validan con Pydantic y los archivos se leen aplicando un límite de tamaño; además, se normalizan los nombres y se evita devolver excepciones internas en los mensajes de error. En cuanto a la clave privada, esta permanece únicamente en la memoria del navegador durante la sesión y no se guarda en el servidor.

La canalización de GitHub Actions instala las dependencias con versiones fijadas, ejecuta las pruebas, inicia Bandit y compila el cliente con Vite. Aunque el análisis estático ayuda a detectar problemas, no cubre por sí solo todos los casos, pues incluso las herramientas evaluadas sobre cambios vulnerables todavía producen falsos positivos y omiten ciertos fallos [17].

Por ello, en este proyecto el escaneo se acompaña de pruebas funcionales, mientras que Trivy y Nmap se dejaron como herramientas adicionales para las prácticas de laboratorio.

V. RESULTADOS

Cuadro II
RESULTADOS EXPERIMENTALES (30 REPETICIONES)

Métrica	Resultado
Tamaño de carga	1 081 344 bytes
Cifrado AES-GCM, media	3.074 ms
Desv. estándar del cifrado	0.767 ms
Descifrado AES-GCM, media	3.008 ms
Verificación RSA-PSS, media	1.516 ms
Detección de alteraciones	100 %
Pruebas automatizadas	140/140
Hallazgos Bandit	0
Vulnerabilidades npm audit	0

La prueba integral reproduce el recorrido principal de un usuario: primero crea una cuenta e inicia sesión; después carga un documento, genera las claves, firma el archivo y verifica la firma; a continuación, emite un certificado, comprueba su validez, lo revoca y confirma que el sistema ya no lo acepte. Asimismo, se verificó que una consulta anónima de documentos devuelva el código HTTP 401. En total, las 140 pruebas finalizaron correctamente y las únicas advertencias provinieron del uso interno de `datetime.utcnow()` en la dependencia `python-jose`, sin afectar el funcionamiento evaluado.

Los datos de la Tabla II muestran que los escenarios básicos se cumplieron, ya que el mismo contenido conserva su resumen, mientras que la modificación de un byte hace que la firma sea rechazada. De igual manera, AES-GCM recuperó el archivo original y los certificados revocados dejaron de considerarse válidos. Aunque los tiempos obtenidos son adecuados para el tamaño de archivo utilizado en el experimento, estos resultados no deben interpretarse como una comparación entre algoritmos ni como una prueba de rendimiento con varios usuarios conectados al mismo tiempo.

VI. VERIFICACIÓN, VALIDACIÓN Y AMENAZAS A LA VALIDEZ

Para que las pruebas representaran los requisitos se utilizaron operaciones criptográficas estandarizadas y escenarios relacionados con las funciones del sistema; aun así, existen varias limitaciones. En primer lugar, las mediciones se realizaron en una sola computadora y pueden verse afectadas por otros procesos del sistema operativo, por lo que se usaron 30 repeticiones y se reportó la desviación estándar del cifrado. Además, la autoridad certificadora se mantiene dentro del proceso de la aplicación para facilitar el prototipo, aunque el estado de revocación sí se guarda en la base de datos. Finalmente, no se probaron HSM, conexiones TLS, múltiples nodos o usuarios concurrentes, de modo que los resultados todavía no representan un despliegue de producción.

La revisión de literatura también presenta límites, debido a que fue planteada como una revisión rápida y no como un

metaanálisis. Además, los resultados disponibles en los índices pueden cambiar y la selección fue realizada por los mismos autores del proyecto, lo que introduce un posible sesgo. Para reducir este riesgo se definieron las preguntas antes de buscar, se conservó la cadena utilizada y se aplicaron criterios explícitos; asimismo, siempre que fue posible se incluyeron referencias con DOI o documentos normativos. En general, los estudios coinciden en la utilidad de automatizar controles, aunque también señalan que una herramienta aislada no reemplaza las decisiones de arquitectura ni la revisión de una persona [2], [13], [17].

VII. CONCLUSIONES

SecureSign cumplió el alcance definido para una plataforma académica de firma digital, pues integra operaciones CRUD, autenticación, cifrado, cálculo de resúmenes, firmas, certificados, revocación y auditoría dentro de una arquitectura que puede desplegarse con Docker. Durante la evaluación se detectaron todas las alteraciones introducidas de forma intencional y, para el archivo probado, los tiempos medios se mantuvieron por debajo de 3.1 ms, mientras que los escaneos realizados no reportaron hallazgos.

La revisión guiada por Kitchenham ayudó a tratar la automatización y la medición como parte del diseño, en lugar de dejarlas para el final; sin embargo, el prototipo todavía puede mejorarse. Entre los siguientes pasos se encuentran trasladar el secreto JWT a un gestor de secretos, habilitar TLS mediante un proxy inverso, cifrar los objetos almacenados y proteger mejor la clave de la autoridad certificadora. También sería conveniente incorporar MFA, rotación de claves, límites de intentos, migraciones con Alembic y pruebas con DAST, Trivy y Kali Linux. Por último, si se buscara dar valor probatorio real a las firmas, harían falta sellos de tiempo, políticas verificables de identidad y claves privadas almacenadas en un HSM o en dispositivos controlados por cada usuario.

REFERENCIAS

- [1] H. Myrbacken and R. Colomo-Palacios, "Devsecops: A multivocal literature review," in *Software Process Improvement and Capability Determination*, 2017, pp. 17–29.
- [2] R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting devsecops: A systematic review," *Information and Software Technology*, vol. 141, p. 106700, 2022.
- [3] X. Zhou *et al.*, "Revisit security in the era of devops: An evidence-based inquiry into devsecops industry," *IET Software*, 2023.
- [4] National Institute of Standards and Technology, "Secure hash standard (shs)," NIST, Tech. Rep. FIPS PUB 180-4, 2015.
- [5] M. S. Turan, E. Barker, W. Burr, and L. Chen, "Recommendation for password-based key derivation: Part 1: Storage applications," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-132, 2010.
- [6] K. Moriarty, "Pkcs #5: Password-based cryptography specification version 2.1," RFC Editor, Tech. Rep. RFC 8018, 2017.
- [7] National Institute of Standards and Technology, "Advanced encryption standard (aes)," NIST, Tech. Rep. FIPS PUB 197-upd1, 2023.
- [8] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac," National Institute of Standards and Technology, Tech. Rep. NIST SP 800-38D, 2007.
- [9] K. Moriarty, B. Kaliski, J. Jonsson, and A. Rusch, "Pkcs #1: Rsa cryptography specifications version 2.2," RFC Editor, Tech. Rep. RFC 8017, 2016.

- [10] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet x.509 public key infrastructure certificate and certificate revocation list (crl) profile," RFC Editor, Tech. Rep. RFC 5280, 2008.
- [11] M. Sánchez-Gordón and R. Colomo-Palacios, "Security as culture: A systematic literature review of devsecops," in *IEEE/ACM 42nd International Conference on Software Engineering Workshops*, 2020, pp. 266–269.
- [12] A. Rahman and L. Williams, "Software security in devops: Synthesizing practitioners' perceptions and practices," in *International Workshop on Continuous Software Evolution and Delivery*, 2019.
- [13] M. A. Akbar *et al.*, "Identifying the primary dimensions of devsecops: A multi-vocal literature review," *Journal of Systems and Software*, vol. 214, p. 112063, 2024.
- [14] B. Kitchenham and S. Charters, "Guidelines for performing systematic literature reviews in software engineering," Keele University and Durham University, Tech. Rep. EBSE-2007-01, 2007.
- [15] B. Kitchenham, O. P. Brereton, D. Budgen, M. Turner, J. Bailey, and S. Linkman, "Systematic literature reviews in software engineering—a systematic literature review," *Information and Software Technology*, vol. 51, no. 1, pp. 7–15, 2009.
- [16] M. Jones, J. Bradley, and N. Sakimura, "Json web token (jwt)," RFC Editor, Tech. Rep. RFC 7519, 2015.
- [17] S. Lee *et al.*, "An empirical study of static analysis tools for secure code review," *Proceedings of the ACM on Software Engineering*, pp. 691–703, 2024.